3. Funkcje przeciążone, inline i rekurencyjne - pytania i zadania

A. Pytania kontrolne

1. Na czym polega przeciążenie funkcji?

2. Jakie elementy listy parametrów mogą odróżniać funkcje przeciążone?

3. Dlaczego nie można przeciążać funkcji wyłącznie typem zwracanym?

4. Podaj przykład sytuacji, w której przeciążenie funkcji poprawia czytelność programu.

5. Czym jest funkcja inline?

6. Czy słowo inline gwarantuje, że kompilator wstawi kod funkcji w miejscu wywołania? Wyjaśnij.

7. Czym jest rekurencja?

8. Dlaczego funkcja rekurencyjna musi mieć przypadek bazowy?

9. Czym różni się rekurencja bezpośrednia od pośredniej?

B. Analiza kodu

1. Która wersja funkcji zostanie wywołana?

```
int suma(int a, int b);
double suma(double a, double b);

cout << suma(2, 3);
cout << suma(2.5, 3.5);
```

3. Podaj kolejne wywołania funkcji nwd(18, 12).

```
int nwd(int a, int b) {
    if (b == 0) return a;
    return nwd(b, a % b);
}
```

C. Zadania programistyczne

1. Napisz przeciążone funkcje pole(): dla kwadratu, prostokąta i trójkąta. Każda wersja powinna mieć odpowiednią listę parametrów.

2. Napisz przeciążone funkcje maksimum(): dla dwóch liczb całkowitych, trzech liczb całkowitych oraz dwóch liczb rzeczywistych.

3. Napisz funkcję inline obliczającą sześcian liczby.

4. Napisz rekurencyjną funkcję potega(a, n), która oblicza a podniesione do potęgi n dla n >= 0.

5. Napisz rekurencyjną funkcję sumaOd1DoN(n), która zwraca sumę liczb od 1 do n.

D. Odpowiedzi skrócone

1. Przeciążenie to ta sama nazwa funkcji i różne parametry. Różnić mogą się liczba, typy lub kolejność parametrów. Nie można przeciążać wyłącznie typem zwracanym, ponieważ sam typ wyniku nie musi wynikać z wywołania. inline jest sugestią dla kompilatora. Rekurencja to wywołanie funkcji przez samą siebie; wymaga przypadku bazowego, aby nie działała bez końca. nwd(18,12): nwd(12,6), nwd(6,0), wynik 6.